

FACULTAD DE INGENIERÍA

CURSO: DISEÑO DE PATRONES

SISTEMA DE RESERVAS Y CONTROL DE HABITACIONES DE UN HOTEL

Proyecto básico en Java 17, Swing y Apache ANT

Integrantes: [Completar apellidos, nombres y códigos]

Docente: [Completar]

Ciclo: [Completar]

Año: 2026

Enlace GitHub: <https://github.com/T1T0-1/HotelReservasPatrones>

Nota: el enlace quedará activo después de crear el repositorio y subir el proyecto en la cuenta T1T0-1.

Resumen del proyecto

El proyecto implementa una aplicación de escritorio para administrar habitaciones y reservas de un hotel. **Su alcance es intencionalmente básico:** registrar y eliminar habitaciones, crear reservas, cancelarlas, finalizar estadias, calcular el total, persistir la información en archivos CSV y generar un reporte general. La estructura fue diseñada para evidenciar los cinco principios SOLID, seis patrones GoF y al menos dos principios GRASP.

Elemento	Implementación mínima
Plataforma	Java 17 + Swing
Proyecto	Apache NetBeans con ANT
Persistencia	Archivos CSV y auditoría en archivo de texto
Flujo principal	Habitación disponible -> reserva -> habitación reservada -> cancelación/finalización -> habitación disponible
Pruebas	JUnit 5, Mockito, integración CSV y prueba rápida ANT
Repositorio	https://github.com/T1T0-1/HotelReservasPatrones

1. Problema, objetivo y alcance

1.1 Problema

Un hotel pequeño necesita controlar la disponibilidad de sus habitaciones y registrar reservas sin usar un sistema complejo. El manejo manual puede provocar reservas duplicadas, pérdida de datos y dificultad para conocer la ocupación.

1.2 Objetivo general

Desarrollar un sistema Java básico que controle habitaciones y reservas, con persistencia, manejo de errores, pruebas y evidencias claras de SOLID, GoF y GRASP.

1.3 Alcance funcional

- Registrar, consultar y eliminar habitaciones disponibles.
- Crear reservas validando datos, fechas y disponibilidad.
- Cancelar una reserva o finalizar una estadía.
- Liberar automáticamente la habitación cuando corresponda.
- Calcular el total según el tipo de habitación y número de noches.
- Guardar habitaciones y reservas en archivos CSV.
- Generar un reporte simple de ocupación e ingresos.
- Registrar eventos de reserva en data/auditoria.log.

2. Arquitectura y flujo end-to-end

La solución usa una separación por capas para mantener el código simple y evitar una clase que haga todo:

Capa	Responsabilidad	Clases principales
Presentación	Interfaz Swing y captura de acciones.	MainFrame
Control	Recibe solicitudes de la vista y delega.	HotelController
Aplicación	Ejecuta casos de uso y reglas.	HabitacionService, ReservaService, ReporteService
Dominio	Entidades y estados del negocio.	Habitacion, Reserva, enums
Infraestructura	Persistencia física en CSV.	LegacyCsvStore y adaptadores CSV
Configuración	Construye e inyecta dependencias.	AppConfig

Flujo principal: el usuario registra o selecciona una habitación disponible, introduce los datos de la reserva, el servicio valida las fechas, selecciona la estrategia de tarifa, construye la reserva, actualiza la habitación, guarda ambos archivos y publica un evento de auditoría.

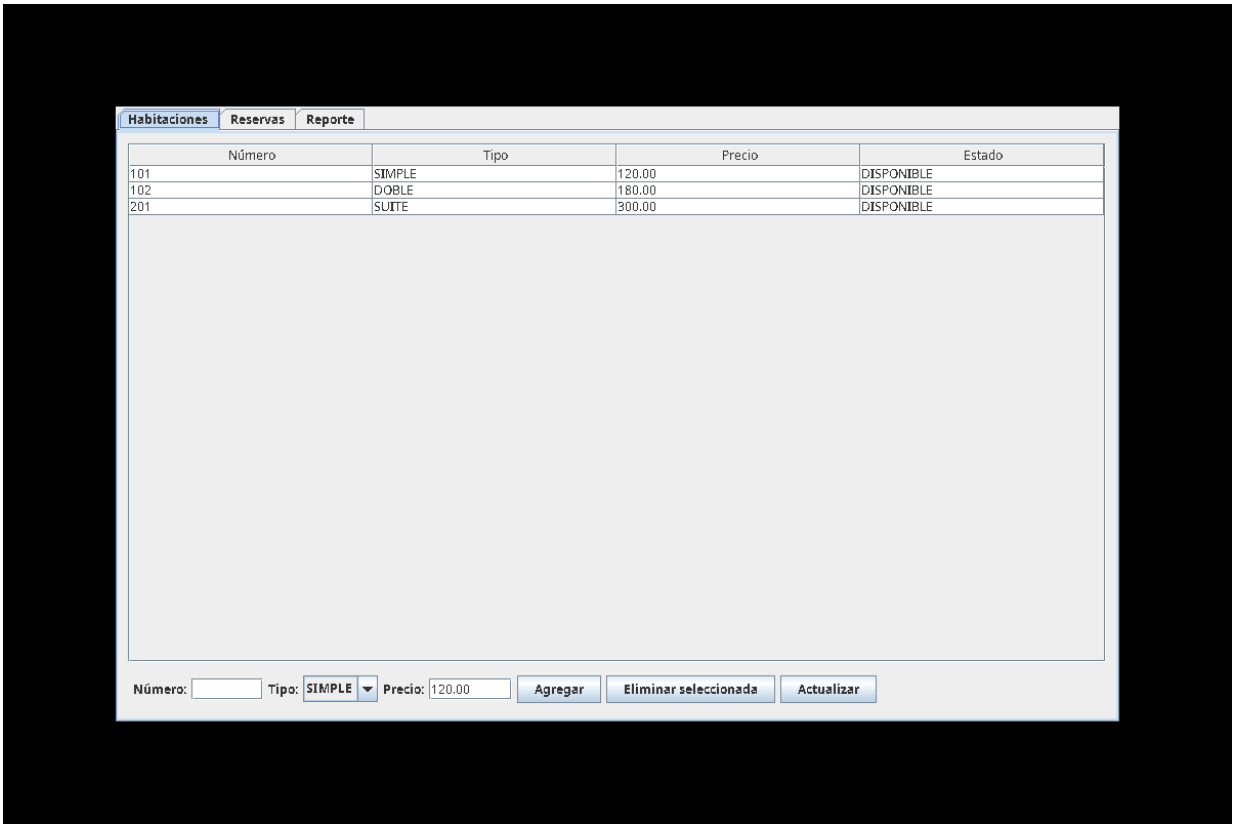


Figura 1. Ventana principal del sistema con datos semilla.

3. Evidencias de los principios SOLID

Principio	Evidencia en el proyecto	Justificación breve
SRP	MainFrame, ReservaService, ReporteService y adaptadores CSV.	Cada clase posee una responsabilidad principal: vista, reservas, reportes o persistencia.
OCP	TarifaStrategy y fábricas concretas.	Se pueden agregar nuevas tarifas creando clases nuevas sin modificar el cálculo central.
LSP	TarifaSimpleStrategy, TarifaDobleStrategy y TarifaSuiteStrategy.	Todas sustituyen correctamente a TarifaStrategy y respetan el mismo contrato.
ISP	HabitacionReadRepository/ HabitacionWriteRepository y equivalentes de Reserva.	Lectura y escritura se separan en interfaces pequeñas.
DIP	HabitacionService y ReservaService reciben interfaces por constructor.	La lógica no depende directamente de archivos CSV; depende de abstracciones.

4. Patrones GoF aplicados

Categoría	Patrón	Clases	Uso
Creacional	Builder	ReservaBuilder	Construye una reserva paso a paso evitando un constructor extenso.
Creacional	Factory Method	TarifaFactory y fábricas concretas	Crea la estrategia adecuada según el tipo de habitación.
Estructural	Adapter	CsvHabitacionRepositoryAdapter y CsvReservaRepositoryAdapter	Adapta LegacyCsvStore a las interfaces Repository.
Estructural	Facade	HotelFacade	Ofrece a la interfaz operaciones simples para habitaciones, reservas y reportes.
Comportamiento	Strategy	TarifaStrategy e implementaciones	Permite intercambiar la regla de cálculo de tarifa.
Comportamiento	Observer	ReservaEventPublisher y AuditObserver	Registra eventos sin acoplar ReservaService al archivo de auditoría.
Extra	Command	CancelarReservaCommand	Encapsula la acción de cancelar una reserva.

Criterio de selección: los patrones se utilizaron únicamente donde resuelven un problema concreto: creación, integración con CSV, simplificación de la vista, cálculo variable y notificación de eventos.

5. GRASP mapeado

Principio GRASP	Evidencia	Resultado
Controller	HotelController recibe los eventos de MainFrame y delega a HotelFacade.	Evita colocar la lógica del negocio en los botones de la interfaz.
Information Expert	Reserva.calcularNoches() y Habitacion administra su disponibilidad.	La responsabilidad queda en el objeto que posee la información.
Low Coupling	Los servicios dependen de interfaces y no de CSV.	La persistencia puede reemplazarse con menor impacto.
High Cohesion	Cada servicio agrupa operaciones relacionadas.	Las clases son pequeñas y fáciles de explicar.

6. Persistencia, errores y pruebas

6.1 Persistencia y datos semilla

Los datos se almacenan en data/habitaciones.csv y data/reservas.csv. El archivo inicial incluye las habitaciones 101, 102 y 201. La escritura se realiza mediante un archivo temporal para reducir el riesgo de pérdida de información.

6.2 Manejo de errores y logging

- Se validan campos obligatorios, fechas y disponibilidad.
- No se permite eliminar una habitación reservada.
- La interfaz muestra mensajes controlados mediante JOptionPane.
- Los eventos se registran en data/auditoria.log y los errores de auditoría usan java.util.logging.

6.3 Pruebas incluidas

Tipo	Clase	Qué comprueba
Unitaria	TarifaStrategyTest	Cálculo del precio por noches.
Unitaria con Mockito	ReservaServiceTest	Guardado de la reserva y actualización de la habitación.
Integración	CsvRepositoryIntegrationTest	Escritura y lectura real de una habitación en CSV.
Integración	FlujoReservaIntegrationTest	Reserva, persistencia, finalización y liberación.
Prueba rápida	SmokeTestRunner	Flujo crear-reservar-cancelar-persistir sin dependencias externas.

Verificación realizada: el proyecto compiló con ANT usando release 17, generó el archivo JAR y superó ant smoke con el mensaje: “SMOKE TEST OK: flujo crear-reservar-cancelar-persistir”.

Comandos: ant clean jar, ant run, ant smoke y ant test. La primera ejecución de ant test requiere internet para descargar JUnit 5 y Mockito.

7. Antipatrones evitados y checklist

Antipatrón	Medida aplicada
God Class	Separación entre vista, controlador, servicios, dominio y persistencia.
Condicionales extensos	Strategy y Factory Method para las tarifas.
Dependencias rígidas	Interfaces Repository e inyección por constructor.
Abuso de Singleton	AppConfig construye objetos explícitamente; no existe estado global.
Lógica en la vista	MainFrame delega en HotelController.
Duplicación	Acceso físico común centralizado en LegacyCsvStore.

Checklist	Estado	Evidencia
Flujo end-to-end funcionando	CUMPLE	ant smoke y servicios del sistema.
SRP, OCP, LSP, ISP y DIP	CUMPLE	Tabla de la sección 3 y clases del proyecto.
2 creacionales, 2 estructurales y 2 de comportamiento	CUMPLE	Tabla de la sección 4.
2 o más GRASP mapeados	CUMPLE	Controller, Information Expert, Low Coupling y High Cohesion.
Pruebas unitarias e integración	CUMPLE	Carpeta test y target ant test.
README y datos semilla	CUMPLE	README.md y carpeta data.
Guion para video demo	PREPARADO	docs/GUIA_VIDEO_DEMO.md; falta grabar y publicar.
Tabla de antipatrones evitados	CUMPLE	Sección 7 y archivo docs/TABLA_ANTIPATRONES.md.
Repositorio GitHub	ENLACE DEFINIDO	Crear el repositorio en T1T0-1 y subir el proyecto completo.

8. Entregables y demostración

- Proyecto completo para NetBeans ANT en la carpeta HotelReservasPatrones.
- README ejecutable, instrucciones, datos semilla y enlace GitHub editable.

- Informe técnico en Word/PDF y versión resumida en Markdown.
- Pruebas unitarias, pruebas de integración y prueba rápida offline.
- Guía de video demo de 5 a 8 minutos.
- Contenido sugerido para una presentación de 10 diapositivas.

8.1 Secuencia breve de demostración

1. Abrir el proyecto en NetBeans y ejecutar.
2. Mostrar las tres habitaciones semilla.
3. Crear una reserva para la habitación 101.
4. Comprobar que cambia a RESERVADA y que se guarda en CSV.
5. Cancelar o finalizar la reserva y comprobar que vuelve a DISPONIBLE.
6. Generar el reporte y mostrar auditoria.log.
7. Ejecutar ant smoke y explicar las tablas SOLID/GoF/GRASP.

Conclusiones

- El sistema cubre un flujo completo con persistencia y manejo básico de errores.
- La aplicación cumple el mínimo de seis patrones GoF sin introducir complejidad innecesaria.
- La separación por capas permite evidenciar los cinco principios SOLID y varios principios GRASP.
- La prueba rápida confirma el flujo principal; las pruebas JUnit 5 y Mockito quedan disponibles mediante ant test.
- Antes de la entrega solo quedan dos acciones externas: subir el proyecto al repositorio indicado y grabar el video demo.

Anexo: enlace de GitHub

Enlace que debe quedar activo antes de entregar: <https://github.com/T1T0-1/HotelReservasPatrones>

El enlace ya está actualizado. Falta crear el repositorio HotelReservasPatrones en la cuenta T1T0-1 y subir el contenido completo del proyecto.